

Đề tài Project

Đề tài 1: Tối ưu hóa bộ nhớ và hiệu năng huấn luyện LLM với DeepSpeed ZeRO-3 và FSDP

Gợi ý:

- Sử dụng môi trường tối thiểu 2 GPU (Kaggle/Colab Pro).
 - Fine-tune mô hình ngôn ngữ (ví dụ: facebook/opt-1.3b hoặc Llama-3.2-1B) trên tập dữ liệu wikitext.
 - So sánh giữa **DeepSpeed ZeRO-3** (kèm CPU Offloading) và **Fully Sharded Data Parallel (FSDP)** của PyTorch.
 - Phân tích tốc độ huấn luyện (throughput - tokens/sec), mức tiêu thụ bộ nhớ VRAM tối đa và thời gian hội tụ giữa DeepSpeed ZeRO-3 và PyTorch FSDP khi có/không sử dụng CPU Offloading cho tham số/optimizer states.
-

Đề tài 2: Huấn luyện song song Pipeline cho các mô hình vượt kích thước bộ nhớ GPU

Gợi ý:

- Sử dụng môi trường 2 GPU.
 - Fine-tune một mô hình lớn không thể nằm gọn trong 1 GPU (ví dụ: facebook/opt-6.7b hoặc gpt2-xl) bằng kỹ thuật **Pipeline Parallelism**.
 - Đối chiếu **DeepSpeed Pipeline Parallelism** với giải pháp mặc định **torch.distributed.pipeline**.
 - Phân tích hiện tượng “Bubble” trong pipeline, ảnh hưởng của số lượng micro-batches đến hiệu suất và so sánh tốc độ huấn luyện trên 2 GPU (Pipeline) so với 1 GPU dùng Gradient Checkpointing.
-

Đề tài 3: Phân tích hiệu năng và khả năng mở rộng (Scaling) của PyTorch DDP

Gợi ý:

- Sử dụng môi trường 2 GPU trở lên.
 - Huấn luyện mô hình Computer Vision quy mô lớn (ví dụ: Vision Transformer - ViT-Large hoặc ConvNeXt-XL) trên tập dữ liệu CIFAR-100 hoặc một phần ImageNet.
 - Triển khai **Distributed Data Parallel (DDP)** của PyTorch.
 - Đo lường Speedup khi tăng số lượng GPU, đối chiếu với **Định luật Amdahl** và phân tích sự khác biệt về độ chính xác khi thay đổi kích thước Batch size (Global vs Local) kèm Linear Learning Rate Scaling.
-

Đề tài 4: Huấn luyện phân tán hiệu quả với nén Gradient và Error Feedback

Gợi ý:

- Sử dụng môi trường 2 GPU (giả lập bằng thông mạng thấp).
 - Fine-tune mô hình ngôn ngữ vừa phải (ví dụ: `distilgpt2`) trên tập dữ liệu `wikitext-2`.
 - Triển khai nén gradient (**Gradient Compression**) bằng **PowerSGD** hoặc **1-bit Adam** (DeepSpeed) so với huấn luyện FP16 thông thường.
 - So sánh tổng lượng dữ liệu truyền tải qua mạng giữa các GPU và đánh giá ảnh hưởng của việc nén đến độ chính xác cuối cùng của mô hình (Perplexity) cùng thời gian huấn luyện tổng thể.
-

Đề tài 5: So sánh kiến trúc Parameter Server và All-Reduce trong huấn luyện phân tán

Gợi ý:

- Sử dụng cụm tính toán (nhiều CPU/GPU) hoặc giả lập các nút mạng.
 - Huấn luyện mô hình Logistic Regression hoặc Neural Network đơn giản trên tập dữ liệu lớn.
 - So sánh **Apache Spark (MLlib)** đại diện cho luồng dữ liệu tập trung và **PyTorch DDP (All-Reduce)** đại diện cho truyền thông phi tập trung.
 - Đánh giá thời gian huấn luyện khi dữ liệu tăng dần (Data Scaling) và phân tích chi phí truyền thông (Communication overhead) của cơ chế All-Reduce so với việc gửi nhận tham số từ Master node.
-

Đề tài 6: Tối ưu hóa truyền thông cộng tác (Collective Communication) với MPI

Gợi ý:

- Sử dụng môi trường hỗ trợ thư viện MPI (cụm HPC hoặc máy tính cài OpenMPI).
- Cài đặt thuật toán nhân ma trận phân tán hoặc cập nhật trọng số Gradient Descent thủ công.
- So sánh việc sử dụng các hàm truyền thông điểm-điểm (`MPI_Send/MPI_Recv`) tự xây dựng cơ chế đồng bộ với các hàm cộng tác tối ưu như `MPI_Allreduce` hoặc `MPI_Bcast`.
- Đánh giá hiệu năng dựa trên cấu trúc liên kết mạng (**Topology**: Ring vs Tree) và so sánh thời gian thực thi (Latency vs Bandwidth) khi thay đổi số lượng tiến trình.